

R3: funciones de auditoría con parámetros

Agrupando comandos de information gathering y auditoría defensiva en funciones reutilizables · Duración: 35 minutos

Dra. Perla Marlene Viera González — Programación para Ciberseguridad, AD2026

Trabajo colaborativo: pueden resolver esta actividad en equipo, comentando y ayudándose entre sí. Cada uno debe subir su propia evidencia a Teams.

Objetivo

Al terminar esta actividad podrás agrupar comandos de PowerShell útiles para auditoría y reconocimiento (information gathering) dentro de funciones propias, que reciban parámetros validados y se invoquen desde la terminal — igual que un cmdlet nativo de PowerShell.

Contexto de ciberseguridad: Revisar qué procesos y servicios corren en un equipo es una tarea básica de auditoría defensiva y de reconocimiento (enumeración pasiva) del propio sistema. Convertir esas consultas en funciones reutilizables es exactamente lo que hace un analista cuando automatiza sus auditorías.

Antes de empezar: buenas prácticas de nomenclatura

Elemento	Convención	Ejemplo
Nombre de función	Verbo-Sustantivo, en PascalCase	<code>Get-ProcesosPorUmbral</code>
Nombre de parámetro	PascalCase, descriptivo, sin abreviar	<code>\$UmbralCPU</code>
Variable local	camelCase o descriptiva en minúsculas	<code>\$resultado</code>

Recuerda: *Get-Verb* te muestra los verbos aprobados por Microsoft si tienes dudas de cuál usar (*Get*, *Test*, *Find*, *Show*...).

Parte 1 · Función 1 — Procesos por umbral de CPU

Vamos a crear una función que revise qué procesos están consumiendo más CPU de lo esperado — una forma simple de detectar actividad anómala en un equipo.

PASO 1 Crea el archivo del ejercicio

Abre PowerShell ISE o Visual Studio Code. Crea un archivo nuevo y guárdalo como `Auditoria_TuNombre.ps1` (usa tu nombre para diferenciarlo si trabajan en equipo).

PASO 2 Declara la función con su nombre

Escribe la palabra clave `function`, seguida del nombre en convención Verbo-Sustantivo:

```
function Get-ProcesosPorUmbral {  
  
}
```

PASO 3 Agrega el bloque `param()`

Dentro de las llaves, agrega un bloque `param()` vacío — ahí irá el parámetro:

```
function Get-ProcesosPorUmbra1 {  
    param (  
  
    )  
}
```

PASO 4 Agrega el parámetro `$Umbra1CPU` con sus modificadores

Dentro de `param()`, declara un parámetro llamado `$Umbra1CPU` de tipo `[double]` (para aceptar números con decimales), que NO sea obligatorio y que tenga un valor por defecto de 50:

```
function Get-ProcesosPorUmbra1 {  
    param (  
        [Parameter(Mandatory=$false)]  
        [double]$Umbra1CPU = 50  
    )  
}
```

¿Por qué no obligatorio? Si el usuario no especifica un umbral, la función debe poder ejecutarse igual usando el valor por defecto (50).

PASO 5 Escribe el cuerpo de la función

Dentro de la función (después del bloque `param`), agrega la línea que hace el trabajo real: obtener los procesos y filtrar los que superan el umbral.

```
function Get-ProcesosPorUmbra1 {  
    param (  
        [Parameter(Mandatory=$false)]  
        [double]$Umbra1CPU = 50  
    )  
    Get-Process | Where-Object { $_.CPU -gt $Umbra1CPU } | Select-Object Name, CPU, Id  
}
```

PASO 6 Ejecuta el script y prueba la función

Corre el script completo (F5 en ISE, o botón ▶ en VS Code) para que PowerShell 'aprenda' la función. Luego, en la terminal, invócala:

```
PS> Get-ProcesosPorUmbra1  
PS> Get-ProcesosPorUmbra1 -Umbra1CPU 10
```

Observa: La primera línea usa el valor por defecto (50). La segunda usa el valor que tú indicaste (10) — deberían aparecer más procesos porque el filtro es menos estricto.

Parte 2 · Función 2 — Servicios por estado

Ahora crearás una segunda función para auditar qué servicios están corriendo o detenidos — útil para detectar si un servicio de seguridad fue desactivado, por ejemplo.

PASO 1 Declara la función

```
function Get-ServiciosPorEstado {  
  
}
```

PASO 2 Agrega el parámetro \$Estado, esta vez obligatorio y validado

A diferencia de la función anterior, aquí el parámetro \$Estado Sí será obligatorio, y solo debe aceptar dos valores posibles: Running o Stopped. Usa [Parameter(Mandatory=\$true)] junto con [ValidateSet()]:

```
function Get-ServiciosPorEstado {  
    param (  
        [Parameter(Mandatory=$true, HelpMessage="Escribe Running o Stopped")]  
        [ValidateSet("Running", "Stopped")]  
        [string]$Estado  
    )  
}
```

Recuerda de la sesión pasada: *Mandatory=\$true obliga a enviar el parámetro. ValidateSet limita las opciones válidas. HelpMessage muestra ayuda si el usuario no lo envía.*

PASO 3 Escribe el cuerpo de la función

```
function Get-ServiciosPorEstado {  
    param (  
        [Parameter(Mandatory=$true, HelpMessage="Escribe Running o Stopped")]  
        [ValidateSet("Running", "Stopped")]  
        [string]$Estado  
    )  
    Get-Service | Where-Object { $_.Status -eq $Estado } | Select-Object Name, Status  
}
```

PASO 4 Ejecuta el script completo y prueba la función

Vuelve a correr el script (ahora con las dos funciones dentro) y pruébala:

```
PS> Get-ServiciosPorEstado -Estado Running  
PS> Get-ServiciosPorEstado -Estado Stopped  
PS> Get-ServiciosPorEstado
```

¿Qué debería pasar en la tercera línea? Como \$Estado es obligatorio, PowerShell debe pedirte el valor (o marcar error) — así confirmas que Mandatory=\$true funciona.

Parte 3 • Evidencia y cierre

PASO Final Captura tu evidencia

- Toma una captura de pantalla o fotografía donde se vea tu archivo .ps1 completo (las dos funciones).
- Toma una o más capturas o fotos de la terminal ejecutando ambas funciones con distintos parámetros, mostrando resultados.
- Asegúrate de que en la captura se alcance a leer tu nombre de archivo (Auditoria_TuNombre.ps1).

Entregable: sube a Teams las capturas de pantalla (código + ejecución) de tus dos funciones funcionando. No se requiere reporte adicional.